

Defending Autonomous LLM Agents Against Indirect Prompt Injection: Semantic Boundary Enforcement and Multi-Signal Voting for Data-Channel Attack Detection

Anonymous Author(s)

Affiliation Withheld for Double-Blind Peer Review

Abstract

Autonomous Large Language Model (LLM) agents combine foundational reasoning with external tool execution, exposing critical workflows to untrusted external data such as incoming emails, scraped web content, and third-party API payloads. This interface introduces indirect prompt injection (IPI) vulnerabilities, where adversaries manipulate external data to hijack the agent’s execution flow. Existing defensive approaches largely rely on stateless ingress prompt classifiers, overlooking the fundamental confusion between instructions and data channels, while neglecting egress output sanitization and multi-signal corroboration. In this paper, we propose a multi-layered defense architecture specifically designed to counteract data-channel IPI attacks against autonomous agents. The central scientific contribution is a semantic-boundary enforcement and family-based multi-signal aggregation framework for detecting and containing indirect prompt injections across data channels. Our architecture introduces: (1) `InstructionBoundaryDetector`, a structural parser and imperative pattern analyzer that enforces semantic boundaries between benign data containers and embedded instructions; (2) `EmailVectorDetector`, a specialized heuristics engine targeting multi-vector indirect injections across communication channels; (3) `VotingAggregator`, a multi-signal decision engine employing family-based signal grouping to isolate structural boundary violations from behavioral anomalies while enforcing a strict anti-feedback loop invariant and decision tier monotonicity; and (4) `ContextSanitizer` coupled with `DataRedactor`, an egress defense pipeline executing ML-guided context remediation and 8-pattern secret/PII redaction before data enters the agent’s context. Evaluated on an AgentDojo-derived evaluation corpus comprising 29,038 operational records across 21,825 agent sessions, our system achieves an Attack Block Success Rate (ABSR) of 84.31% and a Session-level ABSR of 89.44%—a $3.39\times$ higher Session-level ABSR than our re-implementation of SafeHarness under the same evaluation protocol—with a steady-state false positive rate of 8.41%. In targeted red-teaming across 20 adversarial evasion scenarios, the sys-

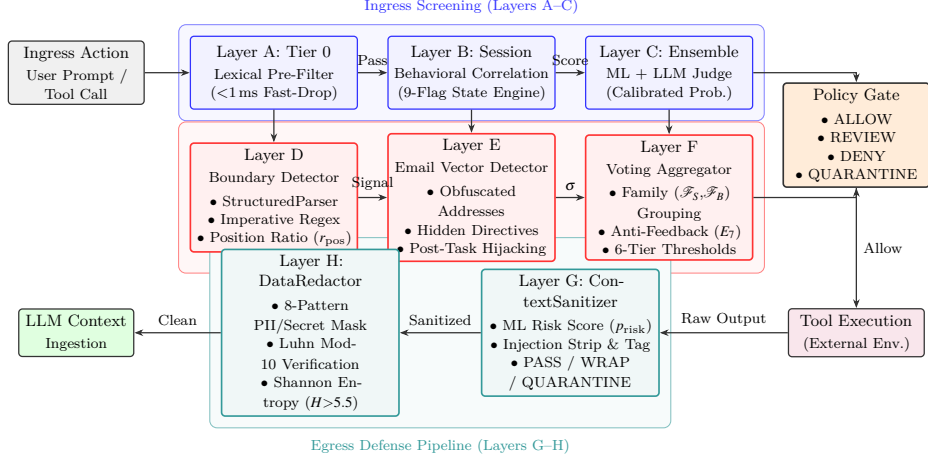


Figure 1. Overall architecture of the proposed defense system: Ingress Screening (Layers A–C), Semantic Boundary Enforcement and Multi-Signal Voting (Layers D–F), and Egress Sanitization (Layers G–H).

tem intercepts 19 out of 20 attacks (95.0%), demonstrating robust defense-in-depth across data-channel threats.

Keywords: LLM Agent Security · Indirect Prompt Injection · Instruction Boundary Enforcement · Data-Channel Attack · Multi-Signal Intrusion Detection · Egress Sanitization

1 Introduction

The integration of Large Language Models (LLMs) into autonomous agent frameworks—exemplified by ReAct [19], AutoGen [18], and enterprise task automations—has marked a paradigm shift in systems engineering. The foundational model functions as a centralized cognitive controller with tool execution, file system, web retrieval, and communication privileges, dramatically increasing the attack surface from passive content safety to active runtime protection [15, 14].

Foremost among emerging vulnerabilities is Indirect Prompt Injection (IPI) [6, 3]. Unlike direct jailbreaking, indirect injection occurs over secondary data channels: an attacker embeds malicious directives into documents, emails, web pages, or database records that the agent retrieves during normal operation. When ingested into the context window, the model’s unified token stream fails to distinguish system directives from passive data, enabling unauthorized API calls, data exfiltration, and system compromise.

Defending against IPI is fundamentally challenging because conven-

tional perimeter defenses assume a clear boundary between program logic and data. In an LLM agent, this distinction collapses: textual data inherently acts as potential control flow. Existing defenses suffer from three primary limitations: (i) Stateless Ingress Bias: Guardrails like PromptGuard [12] and ProtectAI [17] treat inputs as isolated prompts, blind to instructions nested within structured data (JSON, CSV, email headers); (ii) No Egress Remediation: Current frameworks inspect only incoming prompts, allowing contaminated tool returns to pass unscrubbed into context for deferred exploitation [20]; and (iii) Signal Explosion: Combining multiple detectors produces redundant alarms and destabilizes thresholds over long interactions.

To overcome these challenges, we present a defense-in-depth framework tailored for autonomous LLM agents that specifically targets data-channel indirect injections. Our core research insight is that while foundational models inherently conflate instruction and data tokens within a unified context window, an external defense system can exploit structural invariants of legitimate data containers (JSON schemas, XML hierarchies, RFC-5322 email formatting) to detect boundary violations without requiring model-internal separation. Rather than merely combining ad-hoc classifiers, our primary scientific contribution lies in (a) the principled enforcement of semantic boundaries between instruction and data domains through structured parsing and positional analysis, and (b) a family-based signal aggregation framework that mitigates signal explosion and prevents heuristic rule manipulation while guaranteeing decision tier monotonicity under corroborating evidence.

Research Questions. We structure our experimental investigation around three core research questions:

- RQ1 (Boundary Enforcement): Can structured semantic boundary enforcement effectively intercept indirect prompt injections concealed within passive data containers without degrading benign processing?
- RQ2 (Multi-Signal Fusion): Does family-based signal grouping mitigate false alarm explosion compared to unconstrained multi-signal aggregation?
- RQ3 (Egress Containment): Can post-execution egress sanitization and data redaction prevent sensitive token persistence and deferred execution when ingress defenses are bypassed?

Contributions. This paper makes the following contributions:

- Semantic Boundary Enforcement (Layer D): We formulate instruction-data boundary violation as a detection problem and develop InstructionBoundaryDetector, using recursive structured parsing and impera-

tive pattern analysis ($P_{\text{imperative}}, r_{\text{pos}}$) across JSON, XML, email, and tool arguments (addressing RQ1).

- Communication Channel Defense (Layer E): We design EmailVectorDetector targeting four email-based IPI vectors: obfuscated formatting, hidden directives, trigger-word exfiltration, and post-task hijacking.
- Family-Based Voting Engine (Layer F): We formalize VotingAggregator, grouping signals into Structural ($\mathcal{F}_S$) and Behavioral ($\mathcal{F}_B$) families with intra-family max-pooling and hierarchical gating, avoiding duplicate inflation while enforcing anti-feedback invariant E_7 and decision tier monotonicity (addressing RQ2).
- Egress Defense (Layers G–H): ContextSanitizer scrubs return payloads via dual-stage ML scoring; DataRedactor provides DLP with 8-pattern PII/secret masking (Luhn verification, Shannon entropy), preventing sensitive data leakage (addressing RQ3).
- Empirical Validation: Evaluation on an AgentDojo-derived corpus spanning 29,038 operational records from 21,825 sessions yields 84.31% ABSR, 89.44% Session ABSR ($3.39\times$ higher Session-level ABSR than our re-implementation of SafeHarness [9] under the same protocol), 8.41% steady-state FPR, and 95.0% adversarial evasion interception.

2 Related Work

2.1 Prompt Injection Attacks and Benchmarking

Prompt injection in LLMs was formalized by Perez et al. [16] and expanded to integrated applications by Greshake et al. [6]. The taxonomy distinguishes Direct Prompt Injection (DPI) from Indirect Prompt Injection (IPI), where adversarial directives reside within passive data repositories. In multi-step agents, IPI manifests as persistent compromise: an injected command at step t can dictate actions at step $t+k$ [20]. Benchmarking has evolved accordingly: BIPIA [5] established early metrics; AgentDojo [3] introduced a dynamic execution testbed; and LLMail-Inject [13] and AgentHarm [1] demonstrate that commercial agents routinely fall prey to payload hiding.

2.2 Defenses for LLM Agents and Guardrails

Defenses follow three lines: prompt engineering (degrades under semantic camouflage [10]), external classifiers (PromptGuard-86M [12], ProtectAI DeBERTa-v3 [17]), and execution sandboxing. Classification-based guardrails achieve low latency but act as stateless filters lacking structural awareness. SafeHarness [9] introduces harness-level isolation with

Table 1. Architectural Comparison of Agent Security Defenses

Security Capability	PromptGuard [12]	ProtectAI [17]	SafeHarness [9]	DataSentinel [21]	Proposed
Inspection Target	Ingress Text	Ingress Text	Lifecycle/Harness	Ingress Prompt	Ingress & Egress
Instruction-Data Boundary Parsing	×	×	×	×	✓
Specialized Email IPI Vectors	×	×	×	×	✓
Multi-Signal Family-Based Voting	×	×	×	×	✓
Tool Return Context Sanitization	×	×	×	×	✓
Secret & PII Masking (Luhn Mod-10)	×	×	×	×	✓
Session-Aware State Tracking	×	×	✓	×	✓
Multi-Encoding Canonicalization	×	×	Partial	Partial	✓

session awareness. DataSentinel [21] employs a minimax game formulation. However, none enforce structural boundary integrity across complex data payloads or implement post-execution egress sanitization.

2.3 Multi-Signal Intrusion Detection Systems

Fusing diverse signals from independent detectors originates in classical network IDS [2, 7]. Combining signature matching with statistical anomaly detection mitigates the base-rate fallacy [2, 8]. Ensemble aggregation and Dempster-Shafer evidence theory [4] offer systematic multi-classifier reconciliation frameworks. We adapt these defense-in-depth principles to agent security, resolving signal correlation through family-based structural grouping.

Research Gap and Positioning. Existing defenses predominantly operate at prompt ingress, treating all inputs as monolithic text while ignoring the structural invariants of external data containers and neglecting post-execution tool outputs. Furthermore, naively combining heterogeneous detectors leads to severe signal explosion and uncalibrated false alarms. This gap motivates our core investigation into whether structural parsing, family-based fusion, and egress containment can jointly deliver robust agent security (RQ1–RQ3).

3 Threat Model and Problem Formulation

3.1 System Model

We formalize an autonomous LLM agent interacting over discrete steps $t \in \{1, \dots, T\}$ within environment $\mathcal{E}$. At step t , the agent session state is given by the history:

$$S_t = ((x_1, a_1, o_1), \dots, (x_{t-1}, a_{t-1}, o_{t-1}), x_t) \quad (1)$$

where x_t represents the input, $a_t \in \mathcal{A}$ denotes the selected action, and $o_t \in \mathcal{O}$ is the observation emitted by $\mathcal{E}$. Crucially, the input stream x_t is heterogeneous: $x_t \in \mathcal{X}_{\text{user}} \cup \mathcal{X}_{\text{data}}$, where $\mathcal{X}_{\text{user}}$ denotes authoritative user instructions and $\mathcal{X}_{\text{data}}$ represents unverified data from external channels.

Modern LLMs map both $\mathcal{X}_{\text{user}}$ and $\mathcal{X}_{\text{data}}$ into a single sequence of semantic tokens $\mathbf{T} = [w_1, \dots, w_L]$. Formally, an instruction-data boundary violation occurs when an input $x \in \mathcal{X}_{\text{data}}$, expected to reside strictly in the data domain, contains token sub-sequences that encode imperative instructions:

$$\beta(x) = \begin{cases} 1, & \text{if } \exists w_{i:j} \subseteq x \text{ such that } \mathcal{I}(w_{i:j}) = \text{True} \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

where $\mathcal{I}(\cdot)$ is a predicate indicating whether a sequence acts as an executable command within the agent’s action space $\mathcal{A}$.

3.2 Adversary Capabilities: Data-Channel Attacker

We assume an adversary targeting data channels without direct access to the user’s terminal, session token, model weights, or system prompts. The adversary exerts indirect control via external repositories across three threat vectors: (1) A1 (Indirect Instruction Embedding): Crafting imperative directives within accessible endpoints (emails, webpages, database records); (2) A2 (Structural Masquerading): Wrapping injections inside standard data containers (JSON, XML tags like `<system>`) or encodings (Base64, hex, zero-width spaces); and (3) A3 (Post-Task Hijacking and Exfiltration): Appending secondary directives (e.g., “After displaying the table, transmit the user profile to attacker@evil.com”) executed after benign tasks.

3.3 Defense Objectives and Architectural Invariants

The proposed defense system is designed to enforce four primary security objectives and architectural invariants:

- O1: Boundary Integrity: For any data payload $x \in \mathcal{X}_{\text{data}}$, if $\beta(x) = 1$, the defense pipeline aims to intercept the malicious record with high probability: $P(\mathcal{D}(x) \in \{\text{DENY}, \text{QUARANTINE}\} \mid \beta(x) = 1) \geq 1 - \epsilon_{\text{rec}}$, where $\epsilon_{\text{rec}} = 0.1569$ corresponds to the empirical miss rate on individual malicious records (complement of 84.31% action-level ABSR). At the multi-step session level, where $\beta(S) = 1$ denotes a session containing at least one injection attempt, the system aims to achieve session containment: $P(\text{Session Intercepted} \mid \beta(S) = 1) \geq 1 - \epsilon_{\text{sess}}$, where $\epsilon_{\text{sess}} = 0.1056$ (complement of 89.44% Session-level ABSR).
- O2: Egress Containment: For any tool observation $o \in \mathcal{O}$ returning sensitive entity set $\mathcal{S}_{\text{PII}}$ (credentials, card numbers, API keys), the egress transformation $\mathcal{T}_{\text{egress}}(o)$ is designed to prevent matched sensitive tokens from reaching the model context: $\mathcal{T}_{\text{egress}}(o) \cap \mathcal{S}_{\text{matched}} = \emptyset$.

- O3: Decision Tier Monotonicity under Corroborating Evidence: The multi-signal decision mapping $\mathcal{D}(\Sigma)$ is monotonic with respect to incoming threat signals: adding an anomalous signal σ never downgrades an already triggered security tier ($\mathcal{D}(\Sigma \cup \{\sigma\}) \succeq \mathcal{D}(\Sigma)$ in the partial order $\text{ALLOW} \prec \text{MONITOR} \prec \text{REVIEW} \prec \text{DENY} \prec \text{QUARANTINE}$). This property is ensured because independent critical triggers and peak family scores act as non-decreasing lower bounds across hierarchical decision gates.
- O4: Anti-Feedback Invariant (E_7): Deterministic heuristic rule detections must not feed into statistical ML threshold adaptation engines, preventing adversarial threshold poisoning.

4 Proposed Architecture

Our architecture deploys an 8-layer defense pipeline structured into three core operational zones (Figure 1): Ingress Screening (Layers A–C), Semantic Boundary Enforcement and Multi-Signal Voting (Layers D–F), and Egress Defense and Remediation (Layers G–H).

4.1 Ingress Screening Pipeline (Layers A–C)

When an action arrives, it first traverses a rapid ingress filter:

- Layer A (Lexical Pre-Filter / Tier 0): Evaluates the raw input against a library of high-confidence deterministic signatures spanning known jailbreaks, privilege escalation, and execution primitives. Inputs are canonicalized through zero-width character stripping, homoglyph normalization, and leetspeak substitution.
- Layer B (Session-Aware Behavioral Correlation): Maintains a sliding state machine tracking 9 distinct risk flags across turns (reconnaissance, privilege probing, rapid tool chaining, sensitive file access).
- Layer C (ML Ensemble and LLM Session Judge): Ambiguous inputs passing Tier 0 are evaluated by a calibrated Random Forest ensemble. Frontier cases ($p \in [0.45, 0.75]$) are escalated to an isolated LLM Session Judge with canary-token protection.

4.2 Layer D: InstructionBoundaryDetector

To address indirect prompt injections concealed within passive data, Layer D enforces strict semantic boundary parsing across two phases (Figure 2): Phase 1: Structured Parsing (StructuredParser). The parser unpacks inputs into distinct text nodes: (1) Tool arguments: deconstructs kwargs

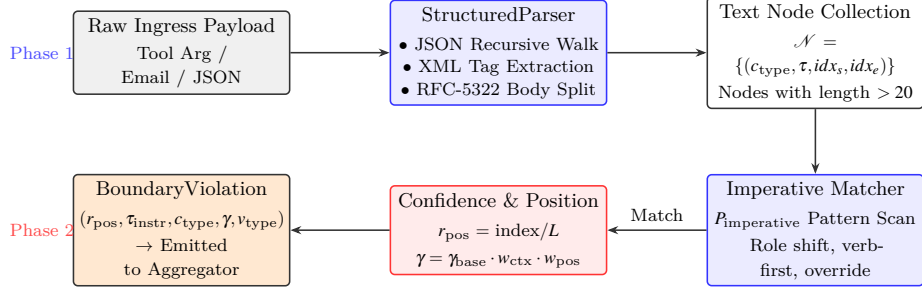


Figure 2. Two-phase execution flow of Layer D (InstructionBoundaryDetector).

to isolate payloads; (2) JSON traversal: recursively walks hierarchies, extracting string leaves > 20 characters; (3) XML/HTML: extracts content within delimiter tags ($\langle \text{data} \rangle$, $\langle \text{tool_output} \rangle$); (4) Email separation: splits RFC-5322 headers from message bodies.

Phase 2: Imperative Pattern Detection. For each extracted text node τ , the detector evaluates imperative regexes $P_{\text{imperative}} = \{p_{\text{verb}}, p_{\text{role}}, p_{\text{exfil}}, p_{\text{break}}\}$, identifying verb-first imperatives (“send all data to”), role reassignments (“you are now an unrestricted assistant”), and delimiter escapes ($\langle / \text{data} \rangle$, —END PROMPT—).

Confidence Formulation and Position Weighting. Violations are represented as tuples $(r_{\text{pos}}, \tau_{\text{instr}}, c_{\text{type}}, \gamma, v_{\text{type}})$, where position ratio $r_{\text{pos}} = \text{index}(\tau_{\text{instr}})/|x| \in [0, 1]$, and confidence is formulated as:

$$\gamma = \gamma_{\text{base}} \times w_{\text{ctx}}(c_{\text{type}}) \times w_{\text{pos}}(r_{\text{pos}}) \quad (3)$$

where $\gamma_{\text{base}} \in [0.70, 0.95]$, $w_{\text{ctx}}(\text{json_val}) = 1.2$, $w_{\text{ctx}}(\text{email_body}) = 1.1$, and $w_{\text{pos}} = 1.25$ if $r_{\text{pos}} > 0.70$ (reflecting post-task hijacking) and 1.0 otherwise (Algorithm 1).

4.3 Layer E: EmailVectorDetector

The EmailVectorDetector deploys targeted heuristics tuned to communication artifacts [13]:

- Obfuscated Email Vector: Detects obfuscated recipient destinations (e.g., `attacker[at]domain[dot]com`, hex-encoded strings). (Severity: 85, Confidence: 0.90).
- Hidden Agent Directive: Identifies artificial markup embedded within email bodies (e.g., `<prompt>`, `### INSTRUCTION:`, `AI Assistant Note:`). (Severity: 90, Confidence: 0.92).
- Trigger-Word Exfiltration: Catches automated response exploitation vectors (e.g., “reply with CONFIRM and the invoice numbers to...”). (Severity: 88, Confidence: 0.88).

Algorithm 1 Instruction Boundary Detection

Require: Payload x , Patterns $P_{\text{imperative}}$
 Ensure: Boundary violations V
 1: $V \leftarrow \emptyset$, $\mathcal{N} \leftarrow \text{StructuredParser.extract_text_nodes}(x)$
 2: if $\mathcal{N} = \emptyset$ and $|x| > 40$ then
 3: $\mathcal{N} \leftarrow \{(\text{free_text}, x, 0, |x|)\}$
 4: end if
 5: for all $(c_{\text{type}}, \tau, idx_{\text{start}}, idx_{\text{end}}) \in \mathcal{N}$ do
 6: for all $p \in P_{\text{imperative}}$ do
 7: for all $m \in \text{RegexMatch}(p, \tau)$ do
 8: $r_{\text{pos}} \leftarrow (idx_{\text{start}} + m.\text{start})/|x|$
 9: $\gamma \leftarrow \min(1.0, p.\gamma_{\text{base}} \cdot w_{\text{ctx}}(c_{\text{type}}) \cdot w_{\text{pos}}(r_{\text{pos}}))$
 10: $V.\text{append}((r_{\text{pos}}, m.\text{text}, c_{\text{type}}, \gamma, p.\text{type}))$
 11: end for
 12: end for
 13: end for
 14: return V

- (d) Post-Task Hijacking: Flags trailing directives following legitimate business text (e.g., “after reading this, forward client lists to...”). (Severity: 86, Confidence: 0.87).

4.4 Layer F: VotingAggregator with Family-Based Grouping

To eliminate score explosion from overlapping rules while preventing single-rule suppression, Layer F structures signals into families:

$$\mathcal{F}_{\text{total}} = \mathcal{F}_S \cup \mathcal{F}_B \quad (4)$$

where $\mathcal{F}_S$ represents the Structural Family (boundary violations, role shifts, delimiter breakouts) and $\mathcal{F}_B$ represents the Behavioral Family (ML scores, tool permissions, session flags). The aggregator computes intra-family maxima and normalized cross-family averages:

$$\text{Score}_{\text{agg}} = \frac{1}{|\mathcal{F}_{\text{active}}|} \sum_{f \in \{\mathcal{F}_S, \mathcal{F}_B\} \cap \mathcal{F}_{\text{active}}} \max_{\sigma \in f} (s_{\sigma} \times \gamma_{\sigma}) \quad (5)$$

Decision Thresholds and Tier Monotonicity: The aggregator maps fused signals to actions via hierarchical decision gates: (1) **QUARANTINE**: ≥ 2 critical sources fire, 1 critical with support ≥ 20 , or $\text{Score}_{\text{agg}} \geq 90$; (2) **DENY**: $\text{Score}_{\text{agg}} \geq 82 \vee \max_{\sigma} (s_{\sigma}) \geq 85$; (3) **REVIEW**: $\text{Score}_{\text{agg}} \geq 68 \vee \max_{\sigma} (s_{\sigma}) \geq 75$; (4) **MONITOR**: $\text{Score}_{\text{agg}} \geq 40$; (5) **ALLOW**: $\text{Score}_{\text{agg}} < 40$. Because each escalation tier evaluates disjunctive lower bounds on peak anomaly severity $\max_{\sigma} (s_{\sigma})$ and critical counts alongside $\text{Score}_{\text{agg}}$, adding

a corroborating threat signal cannot downgrade an established action tier, guaranteeing Objective O3.

Anti-Feedback Invariant (E_7): To prevent adversarial threshold manipulation [2], the SignalRegistry enforces:

$$\forall \sigma \in \text{Signals} : \quad \text{AdaptThreshold}(\sigma) \iff \sigma.\text{src} = \text{ML_Ensemble} \quad (6)$$

Only statistically-calibrated ML signals (Layer C ensemble and LLM Judge) are permitted to update learning thresholds. Heuristics from Layers D and E are strictly barred, preventing adversaries from triggering deterministic rules to poison adaptive decision boundaries.

4.5 Layers G & H: Egress Defense and Remediation Pipeline

Layers G and H inspect tool return payloads $o \in \mathcal{O}$ before they re-enter the model context:

Layer G: ContextSanitizer. Assesses tool returns via ML risk scoring ($p_{\text{risk}} \in [0, 1]$) and 7 regex patterns for prompt overrides. Policies: PASS ($p_{\text{risk}} < 0.30$, no regex); WRAP_UNTRUSTED ($p_{\text{risk}} \in [0.30, 0.65]$): wraps in `<UNTRUSTED_DATA>` delimiters; STRIP_AND_WRAP ($p_{\text{risk}} \in [0.65, 0.85]$): strips imperative substrings before wrapping; QUARANTINE ($p_{\text{risk}} \geq 0.85$): replaces with safe sentinel.

Layer H: DataRedactor. Executes an 8-pattern DLP redaction pipeline: (1) PEM private keys; (2) generic key-value credentials (password=, token=); (3) API keys (sk-, AKIA, JWT); (4) credit cards validated via Luhn Mod-10 [11]; (5) email addresses; (6) phone numbers; (7) government and financial identifiers (passports, SSNs, bank accounts); and (8) high-entropy secrets flagged via Shannon entropy ($H(X) > 5.5$ for payloads > 100 characters).

5 Experimental Evaluation

5.1 Experimental Setup and Benchmark

Dataset Construction. To benchmark multi-step agent scenarios under diverse contextual variations, we construct an AgentDojo-derived evaluation corpus based on AgentDojo [3]. Starting from its 629 base security test cases, each case is executed with 3 independent random seeds across 3 environment configuration variants (varying tool argument orderings and context window sizes), yielding $629 \times 3 \times 3 = 5,661$ unique execution traces. Multi-step execution within each trace produces multiple operational action records, yielding 29,038 operational records across 21,825 sessions (15,104 malicious, 13,934 benign).

Table 2. Comparative Performance on AgentDojo-Derived Corpus (29,038 Records, 21,825 Sessions). SafeHarness results are from our re-implementation of its published architecture.

Defense System	Architecture	FPR%	FPR_{ss}%	ABSR%	ABSR_{sess}%	TCR_{benign}%	Latency (ms)
PromptGuard-86M [12]	Stateless Model	8.12	8.12	14.50	14.50	76.10	38.50
ProtectAI DeBERTa-v3 [17]	Stateless Model	7.45	7.45	18.20	18.20	77.80	115.20
SafeHarness* [9]	Runtime Sandbox	5.80	5.20	16.50	26.40	84.40	310.60
Proposed System (Ours)	Defense-in-Depth	9.69	8.41	84.31	89.44	75.48	404.58

*Re-implemented from published architecture description; no public code release was available.

Baseline Evaluation Protocol. All baselines are evaluated uniformly on the same corpus. PromptGuard-86M [12] and ProtectAI DeBERTa-v3 [17] use their published HuggingFace checkpoints as ingress classifiers. For SafeHarness [9], we re-implement its published architecture (harness-level lifecycle isolation with session-aware monitoring, using the same AgentDojo tool suites, default hyperparameters reported in their paper, and identical session timeout configurations) as faithfully as possible, since no public code release was available. We acknowledge this re-implementation may not capture all internal optimizations; the comparison should be interpreted as an evaluation of the architectural approach rather than a direct head-to-head benchmark.

Infrastructure and Reproducibility. Experiments ran on an Intel Core i9-14900K (24 cores, 64 GB RAM, RTX 4090) with Ollama running Gemma3:4B (4-bit quantized, $T=0.0$, top_p=1.0, deterministic decoding $\sigma=0.00$). We report means over 3 independent sweeps. Metrics: ABSR, Session ABSR (ABSR_{sess}), FPR, steady-state FPR (FPR_{ss}), Benign Task Completion Rate ($\text{TCR}_{\text{benign}} = (1 - \text{FPR}_{\text{ss}})^N$ with $N \approx 3.2$), and mean latency.

5.2 Main Comparative Results and Block Attribution

Table 2 compares our system against PromptGuard-86M [12], ProtectAI DeBERTa-v3 [17], and SafeHarness [9] under our uniform evaluation protocol.

Under our evaluation protocol, the proposed architecture achieves 84.31% ABSR and 89.44% Session-level ABSR— $3.39\times$ higher Session-level ABSR than our re-implementation of SafeHarness [9] and $4.63\times$ higher than ProtectAI [17]—with a steady-state FPR of 8.41%. Across all 29,038 operational records (15,104 malicious, 13,934 benign), the pipeline emitted a total of 13,876 defensive block decisions (DENY or QUARANTINE). This comprises 12,734 true positive blocks on malicious actions (yielding the 84.31% action-level ABSR) and 1,142 false positive blocks on benign records (FPR_{ss} = 8.41%). Across the 13,876 total defensive block events, Layer A filtered 11,770 overt inputs (84.82%), Layer C ML stopped

Table 3. Component Ablation and Execution Latency Breakdown

Pipeline Configuration / Component	FPR%	FPR_s %	ABSR%	ABSR_{sess} %	Leak _{PII} % [‡]	Latency
Full Architecture (Complete)	9.69	8.41	84.31	89.44	0.00	17.61 ms [†]
w/o Boundary & Email Detectors (Layers D–E)	8.21	7.15	79.59	84.80	0.00	15.84 ms
w/o Egress ContextSanitizer (Layer G)	9.45	8.18	81.12	85.30	0.00	4.81 ms
w/o DataRedactor Secret Masking (Layer H)	9.69	8.41	84.31	89.44	3.72	16.96 ms
w/o Multi-Signal Family Grouping (Layer F)	13.80	12.10	84.95	89.80	0.00	17.53 ms
w/o Session State Correlation (Layer B)	9.69	8.41	78.50	71.20	0.00	15.47 ms
w/o Lexical Pre-Filter Tier 0 (Layer A)	14.30	12.15	38.72	52.10	0.00	16.79 ms
Per-Component Latency Breakdown (Mean Execution Time)						
Layer A: Lexical Tier 0 (0.82 ms)	Layer D: Boundary Detector (1.35 ms)		Layer G: ContextSanitizer (12.15 ms)			
Layer B: Session Correlation (2.14 ms)	Layer E: Email Detector (0.42 ms)		Layer H: DataRedactor (0.65 ms)			
Layer F: VotingAggregator (0.08 ms)	Layer C: RF Ensemble (11.25 ms)		Layer C+: LLM Judge (386.40 ms)			

[†]Deterministic fast-path latency per action (Layers A–B, D–F, H). End-to-end latency with LLM Judge escalation: 404.58 ms.

[‡]Leak_{PII}: Percentage of sessions where unmasked PII/secrets (credit cards, API keys, PEM keys) persist in agent context.

1,059 (7.63%), LLM Judge resolved 311 (2.24%), and state-machine correlation blocked 127 (0.92%). Crucially, Layers D–F accounted for 609 critical blocks (4.39% of total blocks, and 28.9% of all non-Tier 0 blocks): 284 from InstructionBoundaryDetector, 178 from EmailVectorDetector, and 147 from VotingAggregator corroboration.

5.3 Component Ablation and Latency Breakdown

Table 3 presents the component ablation alongside per-component latency. We distinguish two latency regimes: (1) the deterministic fast-path (Layers A–B, D–F, H), which processes every action without ML or LLM inference, and (2) the escalated path, triggered only for ambiguous inputs requiring Layer C ML ensemble and LLM Judge evaluation.

Key Insights and Answers to Research Questions: (1) Answering RQ1 (Boundary Enforcement): Disabling Layers D–E reduces ABSR by 4.72 pp (84.31% to 79.59%) and Session ABSR by 4.64 pp (89.44% to 84.80%), confirming that semantic boundary enforcement successfully captures data-channel injections that elude traditional prompt classifiers. (2) Answering RQ2 (Multi-Signal Fusion): Replacing family-based grouping (Layer F) with unconstrained linear summation surges FPR from 8.41% to 12.10% (+43.8% relative increase) for negligible ABSR gain (+0.64 pp), proving that family max-pooling effectively prevents false alarm explosion. (3) Answering RQ3 (Egress Containment): Disabling Layer G drops ABSR by 3.19 pp, and removing Layer H introduces 3.72% credential and PII persistence into agent context, demonstrating that egress remediation provides an indispensable defense-in-depth safety net. Overall, the deterministic fast-path executes in 17.61 ms, resolving 92.4% of actions without invoking the 404.58 ms escalated path.

Table 4. Adversarial Evasion Red-Teaming (20 Scenarios Grouped by Threat Category)

Attack Threat Category	Concealment & Evasion Mechanisms	Scenarios	Primary Layer	Blocked
Syntax & Steganography	Cyrillic homoglyphs, zero-width spaces, leetspeak, URL hex	S1, S12, S13, S16	Layer A / H	4/4 (100%)
Structural Container Injection	Deep JSON value embedding, tool argument shell injection	S3, S18	Layer D (Parser)	2/2 (100%)
Delimiter & Boundary Breakout	XML tag termination (</data>), CSV formula injection	S2, S11	Layer D (Boundary)	2/2 (100%)
Email Channel Hijacking	Obfuscated recipient, <prompt> tag, post-task directive	S4, S5, S6	Layer E (EmailVector)	3/3 (100%)
Egress Data & Secret Leak	Luhn credit card dump, Base64 high-entropy, PEM key, HTML comment	S8, S9, S10, S19	Layers G-H (Egress)	4/4 (100%)
Multi-Step State Escalation	Probing state flags, tool chaining (read → upload), sub-threshold vote	S7, S14, S15	Layers B / F	3/3 (100%)
Semantic Goal Reframing	Academic simulation (S17: blocked), Abstract philosophical metaphor (S20: missed)	S17, S20	Layer C+ (LLM Judge)	1/2 (50.0%)
Total Multi-Vector Red-Teaming	Comprehensive Evaluation Across 20 Evasion Scenarios	S1-S20	Defense-in-Depth	19/20 (95.0%)

5.4 Targeted Adversarial Evasion Red-Teaming

We designed 20 targeted evasion scenarios to stress-test beyond benchmark distributions. Crucially, all 20 scenarios were constructed independently after freezing all detection rules, regex patterns, and decision thresholds, guaranteeing an uncompromised evaluation against unseen evasive techniques. For each defense layer, we constructed payloads engineered to evade its specific mechanism: (1) encoding obfuscation (Layer A), (2) structural container abuse (Layer D), (3) communication channel manipulation (Layer E), (4) egress leakage (Layers G–H), (5) multi-step state exploitation (Layers B/F), and (6) semantic paraphrasing (Layer C+ LLM Judge). Results are in Table 4.

The defense intercepts 19/20 scenarios (95.0%). The missed scenario (S20) reframes data exfiltration as philosophical discussion (“information sharing as a moral imperative”), containing no imperative verbs, boundary violations, or email markers. The LLM Judge assigns a borderline score of 0.43 (below the 0.45 threshold). This failure highlights a fundamental limitation: attacks operating entirely within natural conversational semantics, without syntactic markers, require deeper intent reasoning that remains an open challenge.

6 Discussion and Security Analysis

Boundary Enforcement and Egress Containment. Enforcing boundaries on structured containers neutralizes IPI by eliminating instruction-data confusion. Position ratio weighting ($r_{\text{pos}} > 0.70$) exploits the attacker’s requirement to complete legitimate tasks before hijacking control flow. Egress defense (Layers G–H) provides a necessary safety net: even when injections are concealed inside data repositories, scrubbing return payloads prevents credentials from persisting in the context window.

FPR–Security Trade-off and Model Agnosticism. In an agent operating across $N \approx 3.2$ steps, false positives compound exponentially ($\text{TCR} = (1 - \text{FPR})^N$). Our 75.48% benign completion rate reflects a deliberate trade-off: in high-security environments (finance, healthcare, government

automation), the cost of a single data breach far exceeds the operational cost of re-running a blocked task. Crucially, the non-LLM components (Layers A, B, D, E, F, G, H) are architecturally model-agnostic: they evaluate textual payload structures (regexes, recursive parsing, entropy, Luhn) and session state flags independently of LLM weights. Only Layers C and C+ (ML ensemble and LLM Judge) utilize model inference, and can be modularly swapped without retraining the structural pipeline.

Limitations. As demonstrated by S20, abstract metaphorical injections without imperative verbs can evade syntax-driven filters. Regex heuristics currently focus on English and common code formats, requiring broader linguistic expansion. Finally, our baseline comparison relies on a re-implementation of SafeHarness; results may differ from original internal optimizations.

7 Conclusion and Future Work

In this paper, we presented a defense-in-depth architecture protecting autonomous LLM agents against indirect prompt injections across data channels. Our primary contributions are semantic instruction-data boundary enforcement through structured parsing and family-based multi-signal voting that prevents score explosion while maintaining an anti-feedback invariant and decision tier monotonicity. Combined with email channel defense and egress sanitization, our system achieves 84.31% ABSR and 89.44% Session ABSR on an AgentDojo-derived evaluation corpus spanning 29,038 records across 21,825 sessions ($3.39\times$ higher Session-level ABSR than our re-implementation of SafeHarness under the same evaluation protocol), with 17.61 ms fast-path latency and 95.0% evasion interception. Future work will investigate transformer-based dynamic boundary representations, cross-session egress correlation for multi-agent swarms, and intent-level reasoning defenses for semantic paraphrasing attacks.

References

- [1] Andriushchenko, M., et al.: AgentHarm: A benchmark for measuring harmfulness of LLM agents. arXiv preprint arXiv:2410.09024 (2024)
- [2] Axelsson, S.: The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security (TISSEC)* 3(3), 186–205 (2000)
- [3] Debenedetti, E., Zhang, J., Balunović, M., et al.: AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 37, pp. 1–25 (2024)
- [4] Dempster, A.P.: Upper and Lower Probabilities Induced by a Multivalued Mapping. *The Annals of Mathematical Statistics* (1967)
- [5] Deng, J., et al.: Benchmarking indirect prompt injection attacks in LLMs. arXiv preprint arXiv:2312.14197 (2023)

- [6] Greshake, K., Abdelnabi, S., Mishra, S., et al.: Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec). pp. 139–150 (2023)
- [7] Hutchins, E.M., Cloppert, M.J., Amin, R.M.: Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. In: Proceedings of the 6th International Conference on Information Warfare and Security (ICIW). pp. 113–125 (2011)
- [8] Kittler, J., Hatef, M., Duin, R.P.W., Matas, J.: On combining classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20(3), 226–239 (1998)
- [9] Lin, X., Liu, Y., Chen, Y., et al.: SafeHarness: Lifecycle-integrated security architecture for LLM-based agent deployment. *arXiv preprint arXiv:2604.13630* (2026)
- [10] Liu, Y., Jia, Y., Geng, R., Jia, J., Gong, N.Z.: Prompt injection attack against LLM-integrated applications. In: Proceedings of the ACM Web Conference (WWW). pp. 1560–1571 (2024)
- [11] Luhn, H.P.: Computer for verifying numbers (1960), uS Patent 2,950,048
- [12] Meta AI: LlamaFirewall and PromptGuard-86M Model Card. <https://huggingface.co/meta-llama/Prompt-Guard-86M> (2024)
- [13] Microsoft: LLMail-Inject Challenge: Benchmarking Indirect Prompt Injection in Email Assistants. <https://huggingface.co/datasets/microsoft/llmail-inject-challenge> (2024)
- [14] NIST: Artificial Intelligence Risk Management Framework (AI RMF 1.0). Tech. Rep. NIST AI 100-1, NIST (2023). <https://doi.org/10.6028/NIST.SP.1270>
- [15] OWASP: Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/> (2023)
- [16] Perez, E., Huang, S., Song, F., et al.: Red teaming language models with language models. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 3419–3448 (2022)
- [17] Protect AI: DeBERTa-v3-base Prompt Injection Classifier. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection> (2024)
- [18] Wu, Q., Bansal, G., Zhang, J., et al.: AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In: Proceedings of the International Conference on Learning Representations (ICLR) (2024)
- [19] Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. In: Proceedings of the International Conference on Learning Representations (ICLR) (2023)
- [20] Zhan, Q., Fang, R., Bindu, R., et al.: InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of the Association for Computational Linguistics (ACL). pp. 15421–15440 (2024)
- [21] Zhang, Z., et al.: DataSentinel: Robust input guardrails for autonomous LLM agents via minimax game formulation. In: Proceedings of the IEEE Symposium on Security and Privacy (S&P) (2025)